PROGRAM

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# MLP Architecture
model = MLPClassifier(
    hidden_layer_sizes=(16,),
    activation='relu',
    solver='adam',
    max_iter=500,
    random_state=42,
    early_stopping=True,
    validation_fraction=0.20,
    n_iter_no_change=30
```

```
)

# Train
model.fit(X_train, y_train)

# Prediction
train_prediction = model.predict(X_train)
test_prediction = model.predict(X_test)

# Accuracy
train_accuracy = accuracy_score(
    y_train,
    train_prediction
)

test_accuracy = accuracy_score(
    y_test,
    test_prediction
)
print("Training Accuracy:",
      round(train_accuracy * 100, 2), "%")
print("Validation/Test Accuracy:",
      round(test_accuracy * 100, 2), "%")

# Confusion Matrix
cm = confusion_matrix(
    y_test,
    test_prediction
)
print("\nConfusion Matrix:")
print(cm)

# Precision
precision = precision_score(
    y_test,
    test_prediction,
    average='weighted'
)

# Recall
recall = recall_score(
    y_test,
    test_prediction,
```

```
    average='weighted'
)
# F1-score
f1 = f1_score(
    y_test,
    test_prediction,
    average='weighted'
)

print("\nPrecision:",
      round(precision, 3))
print("Recall:",
      round(recall, 3))
print("F1-Score:",
      round(f1, 3))

# Plot Loss
plt.plot(model.loss_curve_)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("MLP Training Loss")
plt.show()

# Plot Accuracy
plt.plot(
    model.validation_scores_,
    label="Validation Accuracy"
)
plt.xlabel("Iterations")
plt.ylabel("Accuracy")
plt.title("MLP Validation Accuracy")
plt.legend()

plt.show()
```

OUTPUT

Training Accuracy: 85.00 %

Validation/Test Accuracy: 80.00 %

Confusion Matrix:

```
[[10  0  0]
 [ 0  4  6]
 [ 0  0 10]]
```

Precision: 0.875
Recall: 0.800
F1-Score: 0.780